esprit Se former autrement HONORIS UNITED UNIVERSITIES	Année Universitaire : 2025-2026 **Applications Web Distribuées**

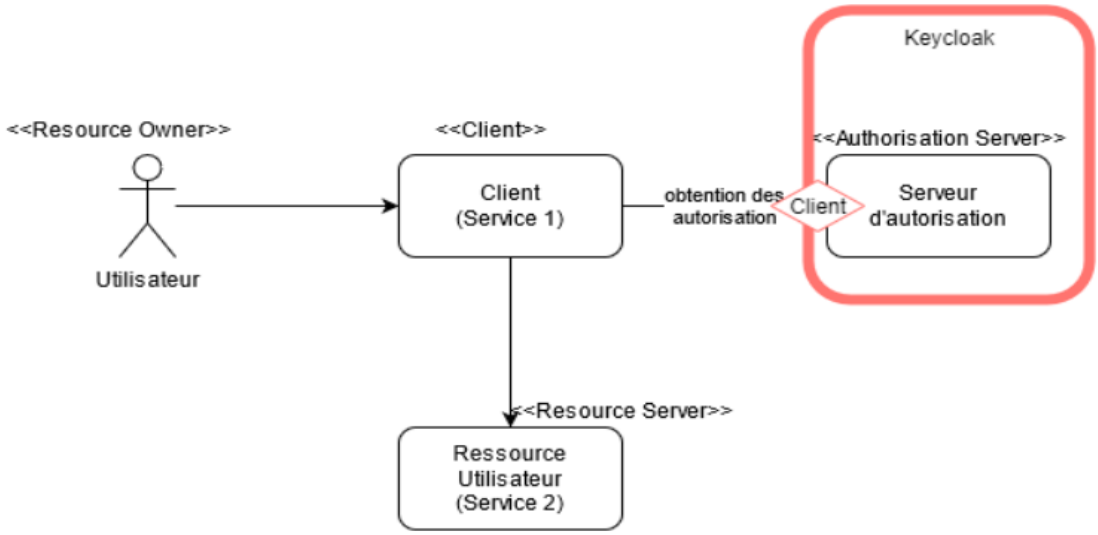
Sécurisation d'un microservice en utilisant « Keycloack »

Objectif

- Intégrer l'aspect sécurité dans l'architecture Microservices en utilisant le serveur d'authentification Keycloack qui fait su SAML ou openconnectID
- Assurer la sécurisation d'un microservice en utilisant Auth2.0.
- Autoriser l'exécution des méthodes par des clients sur leurs rôles

Principe de fonctionnement

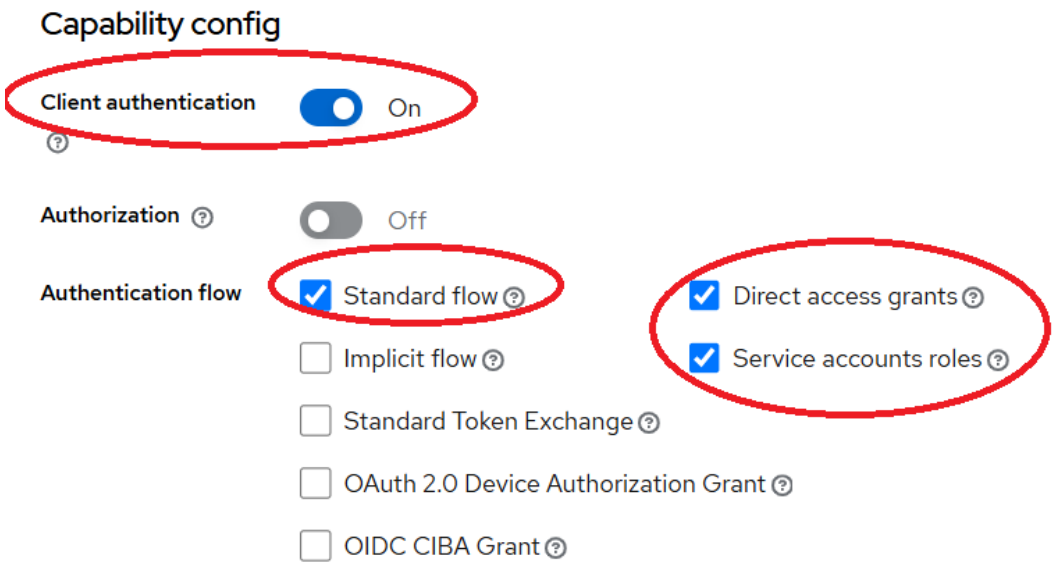
Lors de l'accès à l'application, celle-ci va renvoyer automatiquement l'utilisateur vers Keycloak pour récupérer un Token. Keycloak authentifiera l'utilisateur si besoin, puis renverra des informations sur l'utilisateur et le fameux Token à notre Viewer. Ce Token sera ensuite utilisé pour l'utilisation de Microservice en question.



Les étapes à suivre

Création des clients, des users et attribution des rôles

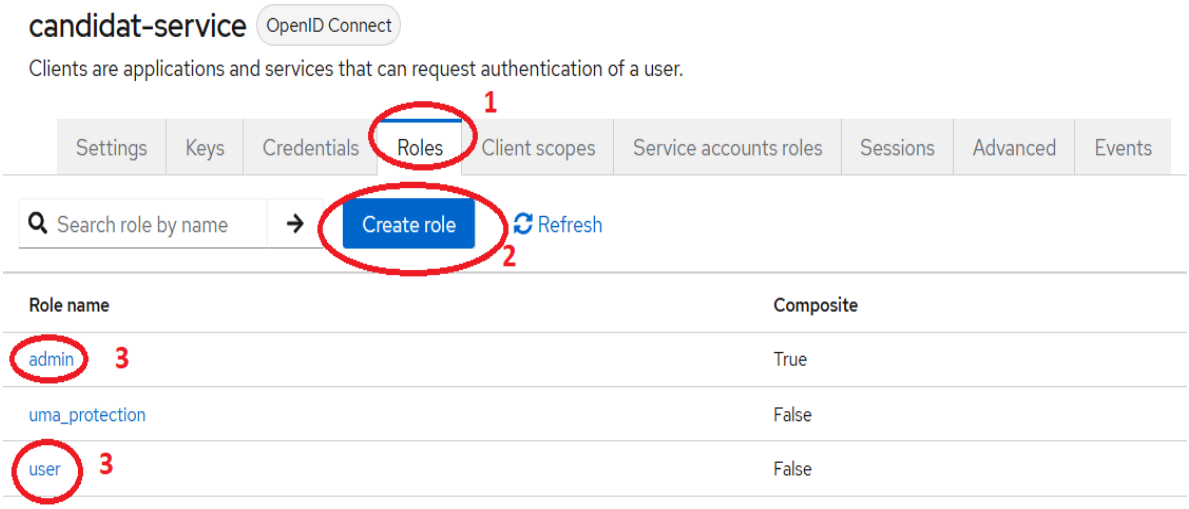
1. Créer un client **« candidat-service »**
   - Pour le client crée, veuillez vérifier cette configuration



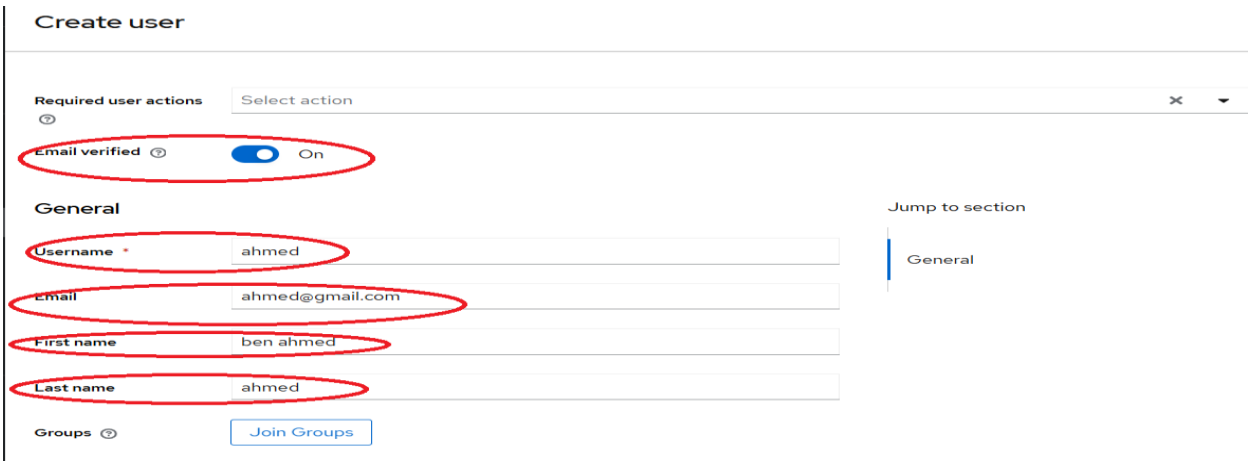
Veuillez noter que :

- **Direct Access Grants** : permet à l'utilisateur final d'obtenir directement un jeton en envoyant son nom d'utilisateur et mot de passe à Keycloak.
- **Standard flow** : C'est le flow standard Authorization Code Flow, utilisé pour les applications **web** ou **SPA** (Single Page Applications). C'est le flow où :
  - L'utilisateur est redirigé vers la page de login Keycloak.
  - Il s'authentifie.
  - Keycloak redirige ensuite ton app avec un "code d'autorisation".
- **Service Accounts roles**, c'est le flow Client Credentials. Il permet à une application elle-même (un client) d'obtenir un jeton sans passer par un utilisateur.

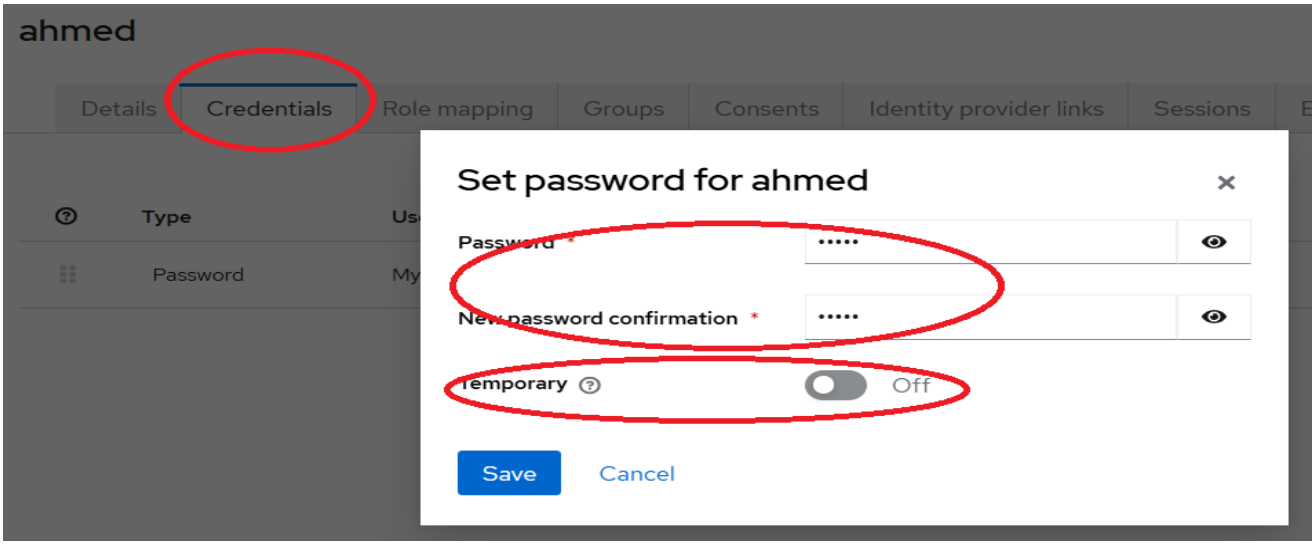
  - Dans l'option Roles, créer deux rôles : user et admin



2. Veuillez choisir « **Users** » et Créer deux utilisateurs et attribuer deux rôles (admin et user) Ahmed a le rôle admin et Amine a le rôle user

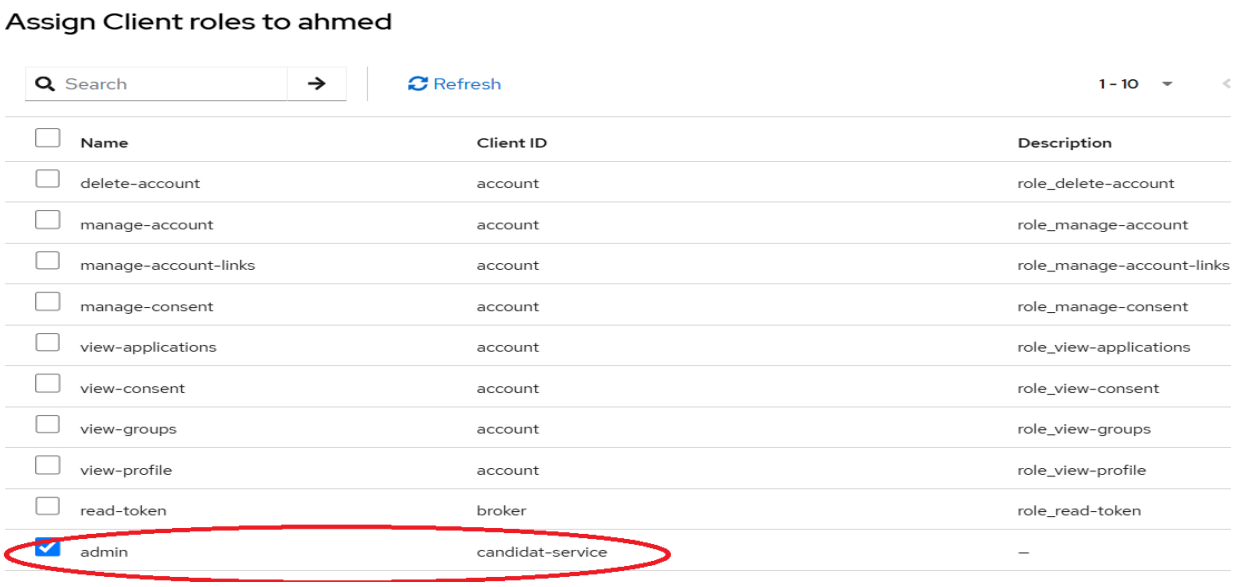


- Dans l'option Credentials, veuillez choisir un mot de passe. Ouis, désactiver l'option Temporary.



3. Etablir la relation entre users et le client Candidat-service

- Veuillez choisir users-> Role Mapping_> Assign Role_> Client Realm. Puis, veuillez choisir comme client ID : candidat-service



4. Configuration Keycloack dans votre Microservice **Candidat**

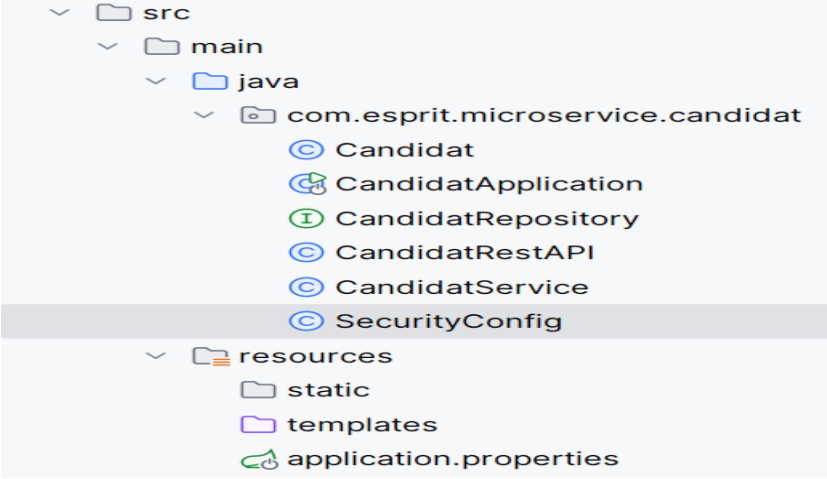
- Dans le fichier pom.xml, veuillez ajouter les dépendances suivantes :

```
- <dependencies>
    <!-- Web + JPA -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- Eureka Client -->
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-
client</artifactId>
    </dependency>

    <!-- Base de données -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Sécurité Spring Boot -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- OAuth2 Resource Server (Keycloak) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-oauth2-resource-
server</artifactId>
    </dependency>

    <!-- OIDC / JWT decoding (nécessaire à Keycloak) -->
    <dependency>
        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-oauth2-jose</artifactId>
    </dependency>

    <!-- Validation -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- Monitoring -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>

    <!-- Lombok (facultatif) -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
```

```
            <optional>true</optional>
        </dependency>

        <!-- Tests -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

-

- Dans le projet Candidat, ajouter la classe **SecurityConfig**



- Dans la classe **SecurityConfig,** veuillez ajouter le code suivant :

```
- package com.esprit.microservice.candidat;
  import org.springframework.context.annotation.Bean;
  import org.springframework.context.annotation.Configuration;
  import org.springframework.security.config.Customizer;
  import
  org.springframework.security.config.annotation.method.configuration.Ena
  bleMethodSecurity;
  import
  org.springframework.security.config.annotation.web.builders.HttpSecurit
  y;
  import org.springframework.security.web.SecurityFilterChain;

  @Configuration
  @EnableMethodSecurity
  public class SecurityConfig {

      @Bean
      public SecurityFilterChain filterChain(HttpSecurity http) throws
  Exception {
          http
                  .csrf(csrf -> csrf.disable())
                  .authorizeHttpRequests(auth -> auth
                          // Autoriser Swagger et Actuator
                          .requestMatchers( "/actuator/**").permitAll()
                          // Toutes les autres requêtes nécessitent un
  token
                          .anyRequest().authenticated()
                  )
                  .oauth2ResourceServer(oauth2 ->
  oauth2.jwt(Customizer.withDefaults()));
```

```
                return http.build();
            }
        }
```

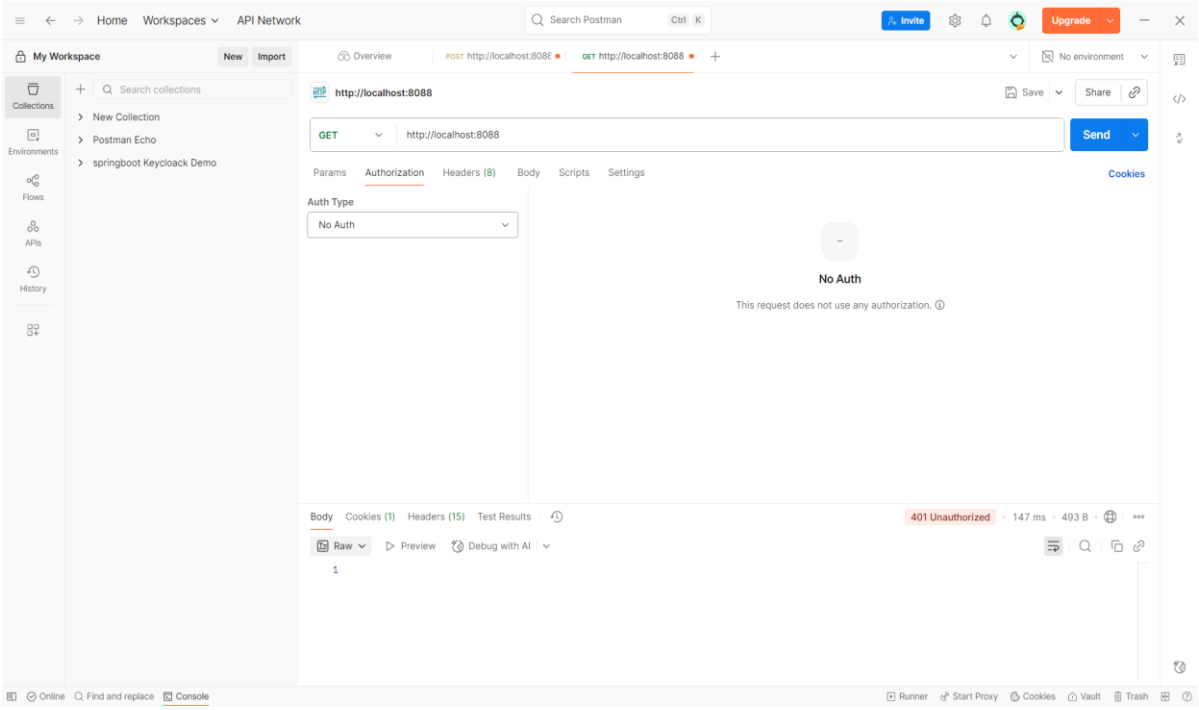
NB: avec un path /actuator, l'accès est limité, nous pourrons accéder au microservice. Mais, nous ne pourrons pas tester les méthodes.

Dans le fichier application.properties, ajouter les propriètés suivantes de keycloakn sachant que **Job_Board_Realm** est le realm crée :

```
spring.application.name=candidat-service
#spring.cloud.config.import-check.enabled=false
#spring.cloud.config.fail-fast=true
server.port=8088
# H2
spring.h2.console.enabled=true
spring.h2.console.path=/h2
#spring.h2.console.settings.web-allow-others=true
# Datasource
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto = create
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
#eureka register
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.client.register-with-eureka=true

# Keycloak realm config
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8080/realms/Job_Board_Realm
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=http://localhost:8080/realms/Job_Board_Realm/protocol/openid-connect/certs
```

1.4 Génération de Token via le postman : Nous allons tester la sécurité de MS-Candidat avec Keycloak

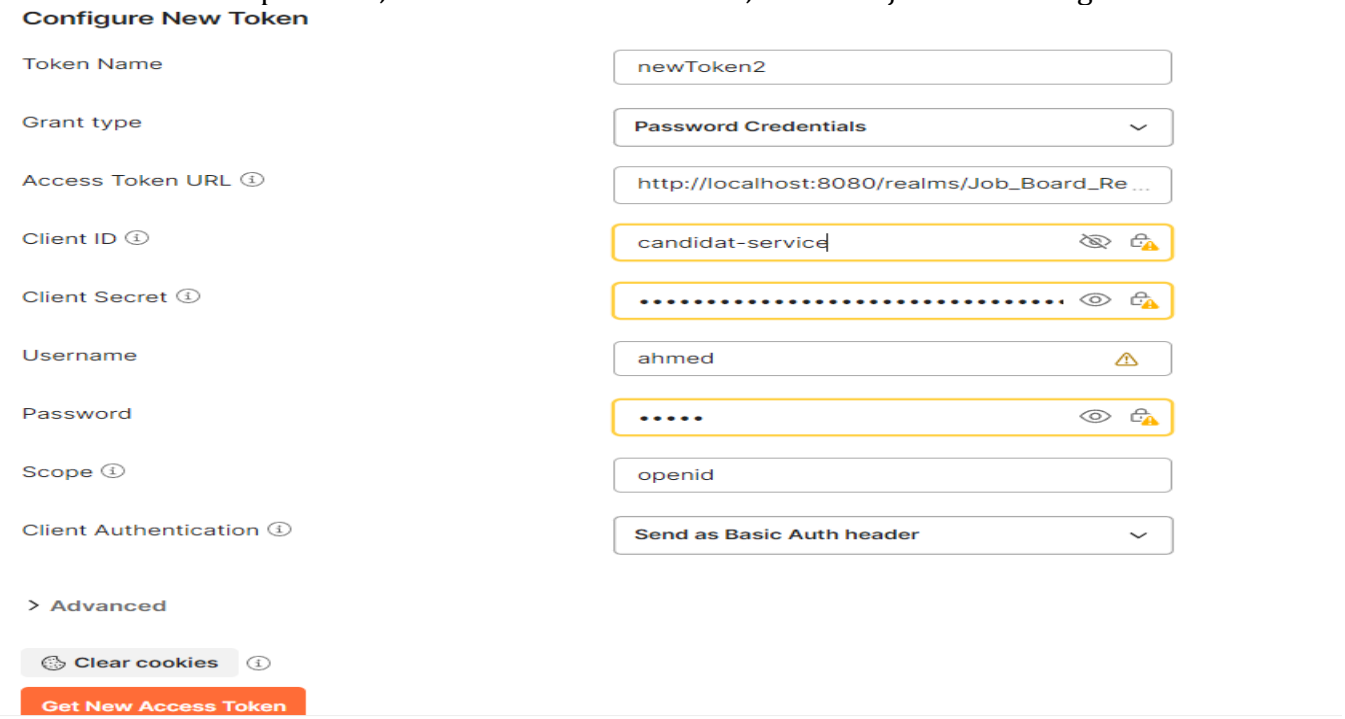
- Sans clé de sécurité, vous devrez voir le statut Unauthorized



- Génération de clé de sécurité :

NB : par défaut Keycloack contient une API qui permet de créer un token à travers la méthode POST

- En utilisant postman, dans une nouvelle fenêtre, veuillez ajouter la configuration suivante :



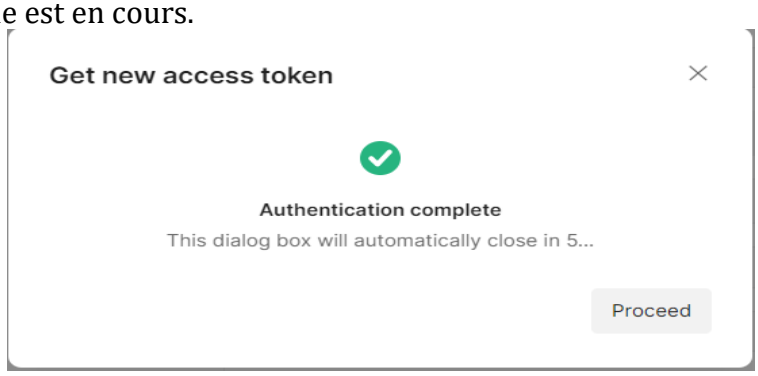
Il faut revenir sur keycloack pour récupérer ces détails :

- Client ID
- Client secret : veuillez le récupérer depuis l'option Crendentials
- Username et password sont les données de l'utilisateur définit en tant que admin (ahmed)
- Pour la valeur du champ Access Token, veuillez suivre les étapes suivantes :

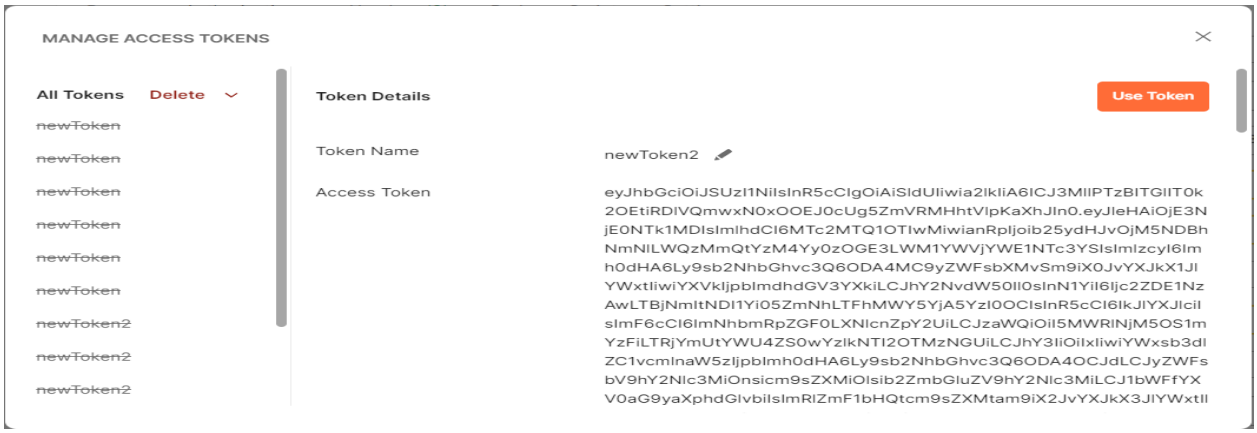
  Realm settings-> cliquer sur la valeur de EndPoints (**OpenID End Point Configuration**) et vous allez voir un résultat comme suit, vous devez sélectionner **la valeur de token_endpoint**

{"issuer":"http://localhost:8080/auth/realms/JobBoardKeycloack","authorization_endpoint":"http://localhost:8080/auth/realms/JobBoardKeycloack/protocol/openid-connect/auth","token_endpoint":"http://localhost:8080/auth/realms/JobBoardKeycloack/protocol/openid-connect/token","introspection_endpoint":"http://localhost:8080/auth/realms/JobBoardKeycloack/protocol/openid-connect/token/introspect","userinfo_endpoint":"http://localhost:8080/auth/realms/JobBoardKeycloack/protocol/openid-connect/userinfo","end_session_endpoint":"http://localhost:8080/auth/realms/JobBoardKeycloack/protocol/openid-connect/logout","jwks_uri":"http://localhost:8080/auth/realms/JobBoardKeycloack/protocol/openid-connect/certs","check_session_iframe":"http://localhost:8080/auth/realms/JobBoardKeycloack/protocol/openid-connect/login-status-iframe.html","grant_types_supported":["authorization_code","implicit","refresh_token","password","client_credentials"],"response_types_supported":["code","none","id_token","token","id_token token","code id_token","code token","code id_token token"],"subject_types_supported":["public","pairwise"],"id_token_signing_alg_values_supported":

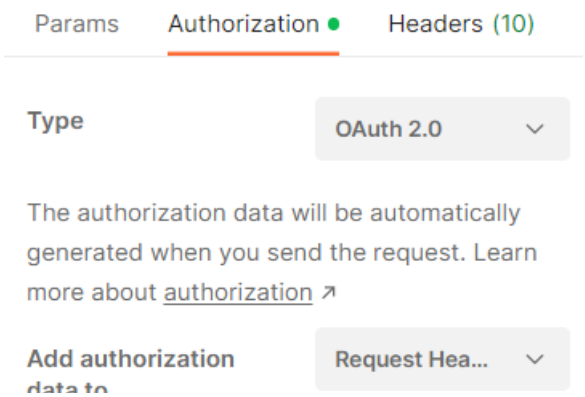
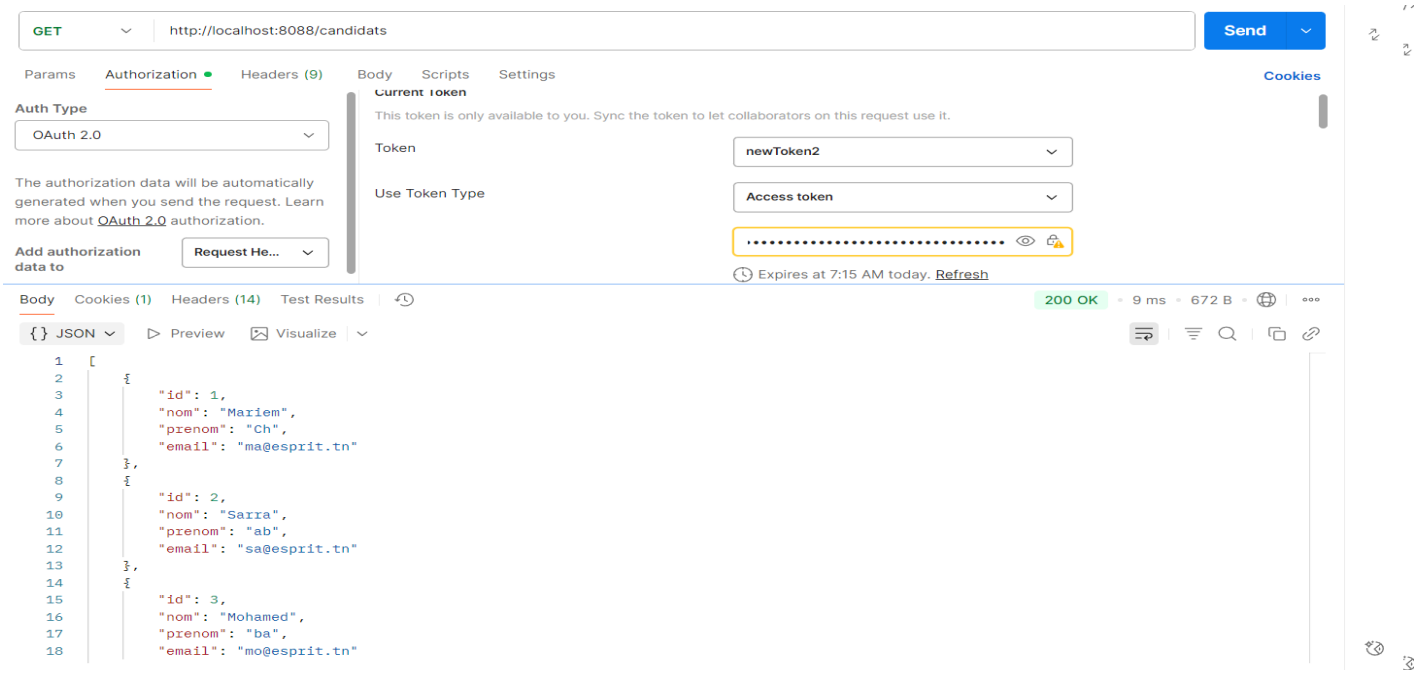
- Maintenant il suffit juste de cliquer sur **Get New Access Token**, vous allez voir que l'Authentification elle est en cours.



- Il suffit de cliquer sur Proceed pour voir la valeur de token et vous allez voir un résultat comme suit. Vous pouvez cliquer sur user Token pour tester la relation sécurité et MS.



- Pour tester vos méthodes, vous devez fixer l'autorisation avec Auth2.0



- Maintenant, nous pouvons tester la méthode GET de notre microservice